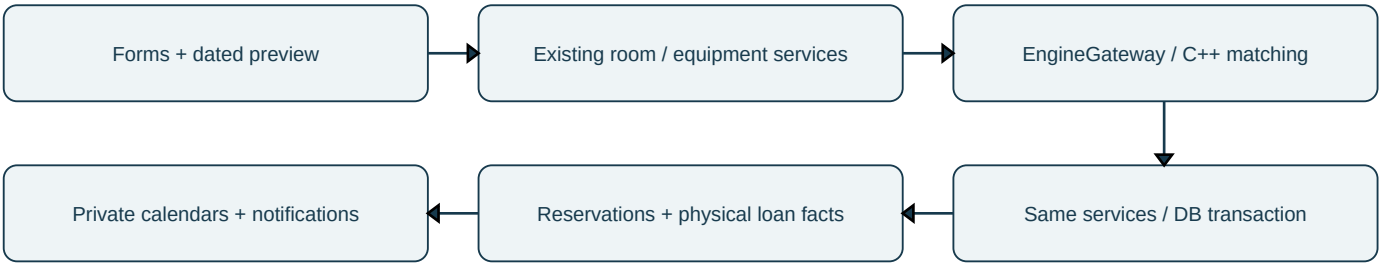
ITERATION 49 / UNIVERSITY RESOURCE ALLOCATION

Recurring Room and Equipment Reservations

Goal: extend UC-STU-02 and UC-STU-03 with explicit bounded weekly series, preserving the existing PostgreSQL, authenticated Flask workflows and C++ allocation engine.

What changed

Students choose weekdays, a date range and a local start/duration, then review every generated occurrence. Room series retain one suitable room across all dates. Equipment recurrence represents separate pickup/return periods, not a continuous loan split into artificial transactions. Faculty/custodian approval confirms the complete feasible proposal or none.



Responsibilities and reuse

Component	Responsibility / change
Shared recurrence utility	New bounded date expansion reuses calendar/timezone helpers. Rejects overlapping, unbounded and ambiguous/nonexistent local times. No allocation decisions.
Existing C++ selection services	Intersect complete room rankings before display truncation; reuse equipment matching for every interval. Existing weekly RecurringRequest and all five academic strategies remain unchanged.
Existing application services	Own authorization, lifecycle, transaction boundaries and exact whole-series approval. Reuse current waiting-list coordinator, staff assignments and events.
Shared record helpers / views	Persist stable series/occurrence identities and retries; project actual dates without creating another booking or calendar system.
Existing frontend / gateway	Reusable weekly selector, signed equipment preview, scoped detail/cancellation controls. Compatible new JSON operations; verified executable and protocol-only stdout remain.

Database additions

Entity	Important information
reservation_series	One unique room-request OR borrowing-request parent; immutable date rule/timezone; retry identity and fingerprint.
reservation_occurrence	Stable ordered start/end instants, parent FK and lifecycle state. Complete expansion is checked at commit.

Existing room reservations/organizer commitments, equipment reservations and extensions gain occurrence links. Request details, participants and quantities are not copied. Schema: 52 application tables plus Alembic; migration head 0049a.

Integrity, Verification and Delivery

Lifecycle and consistency

Rooms: pending series reserve nothing. Current faculty approval rechecks each active date and atomically creates room and organizer commitments. No automatic room switching or partial confirmation.

Equipment: each occurrence identifies reserved assets. Custodians verify collection and physical returns independently. Partial, damaged and overdue returns remain visible; a planned due date never establishes return. Extensions cannot conflict with later reservations.

Cancellation and changes: cancel one future occurrence or remaining future dates. Past room uses and actual handover/return facts stay historical. Active physical loans cannot be released through cancellation. Material changes need a fresh approved proposal.

Concurrency and privacy: reuse existing calendar-state/faculty lock order, revision checks and GiST exclusions. A unique outstanding-unit index prevents duplicate physical possession. Bounded contention returns busy/retry, not a partial result. Server-side participant/staff scope and CSRF remain enabled.

Verification evidence

Check	Actual result
Fresh C++17 full suite	185 passed, 0 failed: 172 existing + 13 new series tests. Two isolated legacy CLI smoke cases passed.
Full Windows Python / real PostgreSQL	453 passed, 2 skipped, 0 failed; 455 discovered tests including 53 new recurrence regressions.
Linux GitHub Actions	455 Python tests passed, 0 skipped/failed; 185 freshly compiled C++ tests passed, 0 failed. PostgreSQL migration, drift, recovery and platform-specific checks passed.
Recovery supplement	Real pg_dump/pg_restore test rerun with matching local tools: 1 passed. Owned temporary restore database removed.
Dates / workflow / concurrency	Multiple weekdays/weeks, DST boundaries, per-date conflicts, all-or-none approval, duplicate submissions, competing threads/processes, optional invitations, cancellations, returns and extensions exercised.
Database / compatibility	Fresh migration, drift check, safe downgrade refusal and one-time upgrade preservation; all five strategy parity checks and existing appointments/calendar/waiting-list/feedback regressions.
Real browser acceptance	1440px desktop and 390px mobile: five room dates, faculty approval/date cancellation; two equipment dates, approval, handover, partial/full physical return; unrelated users denied; no JS errors/page-level overflow.
Documentation / hygiene	One-page UC-STU-02/03 PDFs rendered and reviewed. ER sources/catalog validated against actual models. Credentials, runtime CSVs and test artifacts excluded.

The Windows full run skips the POSIX executable-permission check and recovery when its optional tool path is unset. Recovery was executed separately; Linux CI exercises platform-specific permission behavior and PostgreSQL recovery.

Reviewed decisions and remaining limits

Existing Strategy, Gateway/Adapter and transaction-service boundaries were reused; no new framework or duplicate matching engine. Regression fixes cover professor invitations after cancelling the first date, interval nullability and per-loan overdue alerts. Immutable timezone validation now runs once rather than repeating expensive catalog scans.

Series waiting-list consent/promotion needs an approved policy: current queues remain single-interval and eligible queues are processed before competing approvals. Limits: 366 occurrences, a two-year start-date range and synchronous bounded execution. Legacy academic dates require explicit period bindings. This iteration is not merged or deployed.

Delivery and references

Implementation branch: iteration/49-recurring-reservations. Prerequisite Iteration 48 was merged as 404ed0c. Focused implementation commits: c28fb03, d84afe9, 3ea95a9, 9716c6d, ec6a666, badecf8, c3eb6dd, 49a5587, c11414e. Overdue alerts were also regression-tested against each actual recurring loan due date.

Verified CI: https://github.com/najib-attar/resource-allocation-system/actions/runs/39724228051

Architecture: docs/architecture/recurring-reservations-design.md. Schema: database-design.md and complete ER Mermaid/SVGs. Protocol: docs/engine-protocol.md. Setup and user operations: docs/database-operations.md.